Keyboard and Mouse Driver implementation in mini OS

Github Link:- https://github.com/PSU-CSE597-FA25/project-drivers
Teammates:- Prudhvi Gadupudi (gvp5349@psu.edu)
Sivani Devarapalli (lfd5379@psu.edu)

Legacy PIC Initialization:-

To begin with the project, the assignment code was extended to explicitly support the legacy Programmable Interrupt Controller (PIC) as told in class. The oldo boot-time logic that directly disabled the PIC in the assembly startup code was removed, and PIC initialization was instead implemented by issuing the required ICW commands to the master and slave PICs, remapping their interrupt vectors to 0x20–0x2F. Low-level port I/O helpers were added along with functions to selectively unmask hardware IRQ lines. The kernel initialization sequence was updated to invoke PIC setup before enabling the APIC timer, and the APIC timer interrupt vector and corresponding IDT entry were reassigned to avoid overlap with PIC-generated interrupts. These changes transform the assignment's static interrupt handling into a structured and extensible initialization path.

Keyboard Interrupt Integration:-

After establishing PIC-based interrupt handling, the assignment code was extended to support keyboard input via hardware interrupts (IRQ1). A dedicated keyboard interrupt service routine (ISR) was implemented in assembly to correctly bridge hardware interrupt entry and C-level handling. This ISR saves and restores processor state and invokes a C handler function before returning with iretq. The keyboard interrupt was mapped to vector 0x21 in the Interrupt Descriptor Table using the existing gate setup mechanism, ensuring it does not conflict with CPU exceptions or other device interrupts.

Scancode Reading and Character Output

When a keyboard interrupt occurs, the C-level handler reads the raw scancode from the keyboard controller data port (0x60). Key release scancodes are filtered out, and key press scancodes are translated into ASCII characters using a static lookup table based on scancode set 1. Valid characters are written directly to the framebuffer to provide immediate visual feedback, while special keys such as newline and backspace are handled explicitly to maintain readable console output.

PIC Configuration and Interrupt Acknowledgment

To allow keyboard interrupts to be delivered, IRQ1 was explicitly unmasked on the master PIC after initialization. After each keyboard interrupt is processed, an End-of-Interrupt (EOI) command is sent to the PIC command port to re-enable subsequent interrupts. This ensures correct interrupt flow control at the hardware level and integrates keyboard input cleanly into the existing interrupt infrastructure provided by the assignment code.

Mouse Device Initialization and IRQ Setup

After keyboard support was established, the assignment code was extended to handle PS/2 mouse input using hardware interrupts. The mouse was enabled through the PS/2 controller by issuing the required command sequence to ports 0x64 and 0x60, placing the mouse into streaming mode. The mouse interrupt handler was mapped to interrupt vector 0x2C (IRQ12), and both IRQ12 (mouse) and IRQ2 (PIC cascade) were unmasked to allow mouse interrupts to propagate correctly through the legacy PIC.

Mouse Interrupt Handling and Packet Decoding

Mouse input is delivered as a sequence of three-byte packets, which are assembled incrementally inside the mouse interrupt handler. The handler verifies packet synchronization, extracts relative X and Y movement values, and updates the global mouse position accordingly. Movement values are clamped to screen bounds to prevent invalid framebuffer access. Button state bits are also decoded to detect left, right, and middle mouse clicks. Each interrupt is acknowledged by sending an End-of-Interrupt (EOI) to both the slave and master PICs to maintain correct interrupt flow.

Arrow Cursor Rendering in the Framebuffer

To visually represent mouse movement, the framebuffer code was extended to support rendering a graphical arrow cursor. A fixed bitmap pattern is used to draw the arrow directly onto the framebuffer. Before drawing the cursor, the underlying pixels are saved, allowing the previous cursor position to be erased cleanly when the mouse moves. This approach avoids full screen redraws and minimizes framebuffer updates to only the affected regions.

Separation of Mouse Cursor and Text Cursor

To prevent interference between mouse rendering and text output, the framebuffer logic was refactored to clearly separate the mouse arrow cursor from the text cursor. The text cursor is drawn as a vertical bar aligned to character grid boundaries, while the mouse cursor operates in pixel coordinates. Overlap detection was introduced so that when the mouse cursor intersects the text cursor region, the text cursor is

temporarily hidden and restored after the mouse cursor moves away. This ensures consistent visual behavior during combined keyboard and mouse interaction.

Mouse-Driven Text Interaction and Status Display

Mouse clicks were integrated with the text system by allowing the left mouse button to reposition the text cursor based on the mouse location. This enables typed keyboard input to appear at positions selected using the mouse. Additionally, a mouse status display was added to the framebuffer status area, showing current mouse coordinates and active button states. Together, these changes transform the framebuffer from a purely output-only console into an interactive interface driven by real hardware input.

Outputs:-

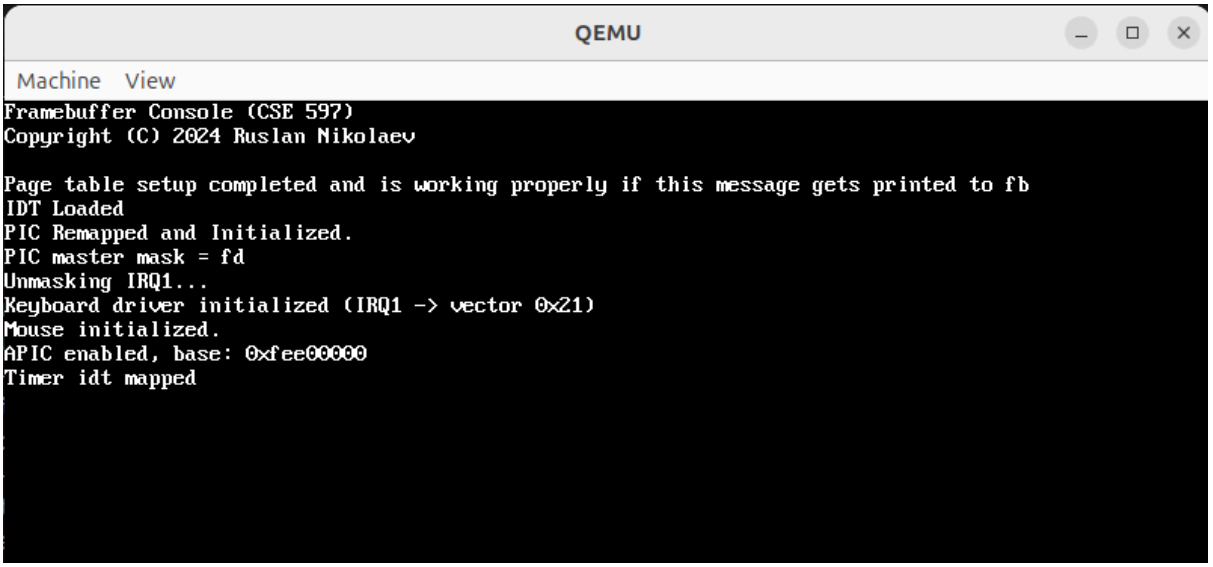

Fig 1:- Figure showing Keyboard and Mouse getting initialized.

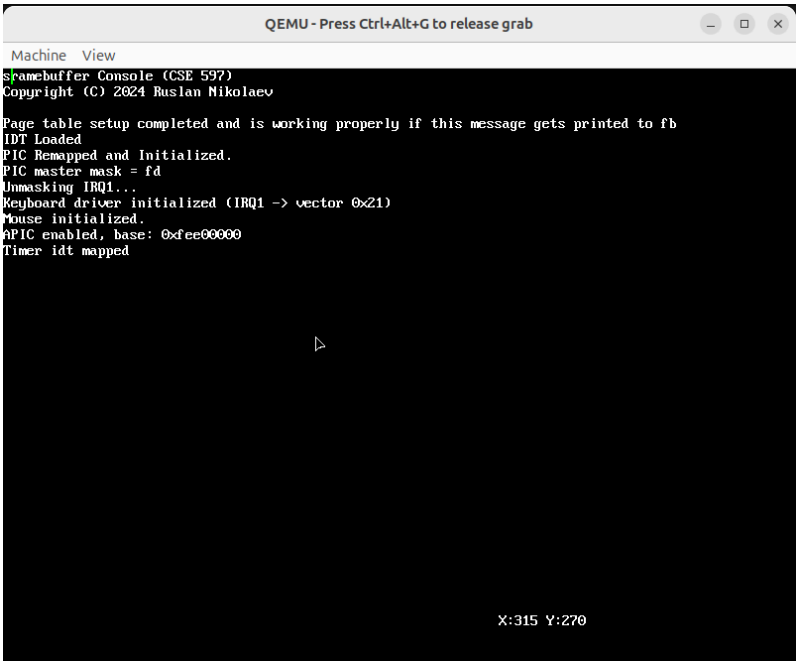

Fig 2:- For showing that we are able to read the input from the mouse we modified our fb.c to show the mouse position at the bottom of the screen always and also we print an arrow shaped text into the framebuffer.

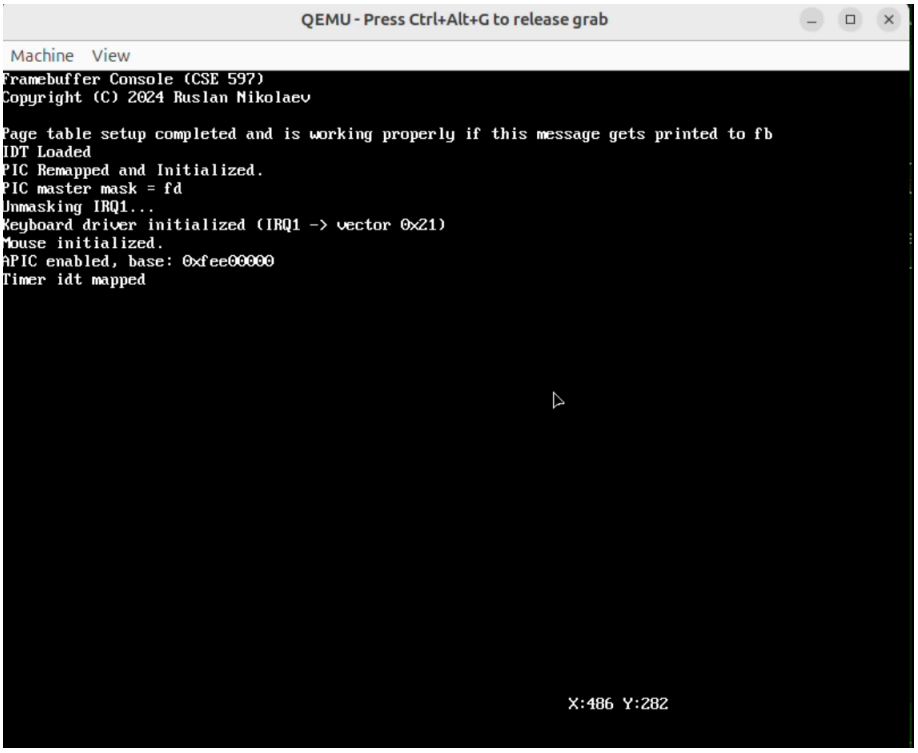

Fig 3:- Showing that values of X and Y are changing when I mouse the cursor and Arrow which is being rendered also is changing its position.

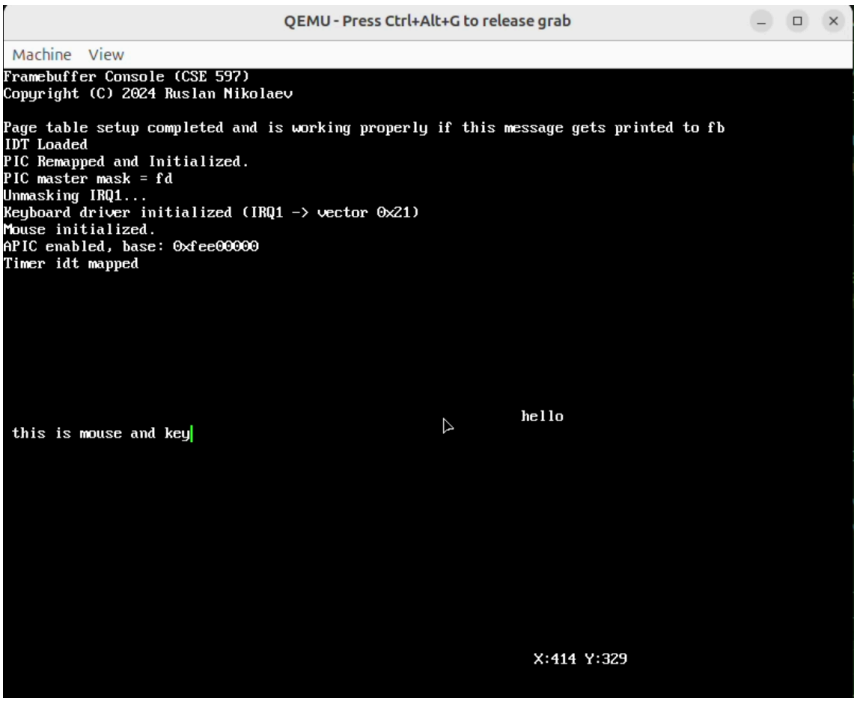

Fig 4:- Showing output from the keyboard which is getting printed while we type, and in the position of mouse cursor.

Fig 5:- Shows keyboard output from multiple different places.

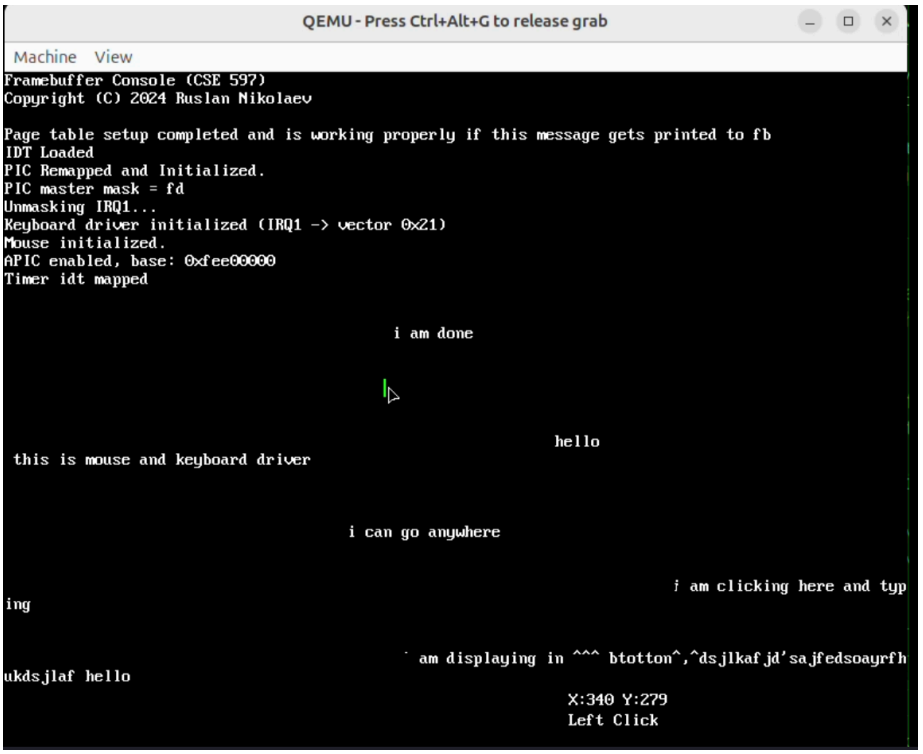

Fig 6:- Shows Mouse Left Click being recorded

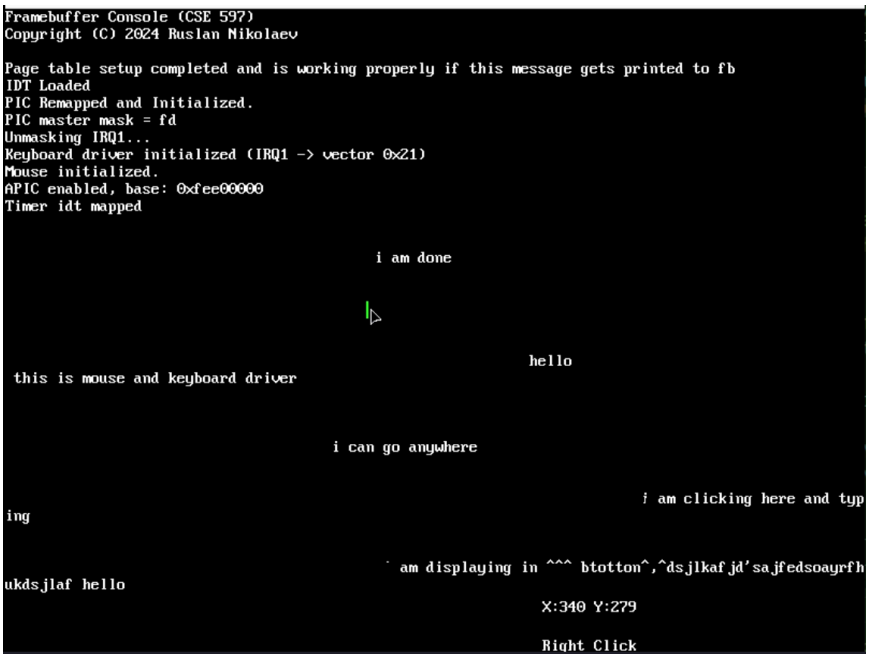

Fig 7:- Shows Mouse Right Click